

Universidade Federal — Disciplina: Redes de Computadores
2022.2

Relatório do Projeto: Chat em Python

Repositório: levisilva22/projeto-redes

Equipe:

- Levi Farias da Silva
- Lucas Guimarães Beelen
- Matheus Almeida

Professor: Almir Pereira Guimarães

Sumário

1	Introdução	2
2	Visão Geral e Arquitetura	2
2.1	server.py (Camada de Conexão)	2
2.2	client.py (Aplicação Cliente)	2
3	Funcionalidades Principais	3
4	Protocolos Implementados	3
5	Dificuldades Encontradas	3
6	O que Poderia Ter Sido Implementado a Mais	4
7	Conclusão	4

1 Introdução

Este documento apresenta o relatório do projeto “Chat em Python”, cujo código-fonte está disponível no repositório `levisilva22/projeto-redes`. O objetivo do trabalho é desenvolver uma aplicação de bate-papo que permita a comunicação entre clientes por meio de uma conexão de rede, demonstrando os conceitos de programação com *sockets* e os principais elementos de protocolos de aplicação.

De acordo com a estrutura atual do repositório, o projeto é escrito 100% em Python e contém, na raiz, os seguintes arquivos principais:

- `client.py` — cliente de chat;
- `server.py` — utilitários de conexão/comunicação em rede;
- `README.md` — instruções e documentação;
- `.gitignore`.

2 Visão Geral e Arquitetura

A aplicação é organizada em módulos Python, com a responsabilidade de conexão de rede separada do código de interação do cliente:

2.1 `server.py` (Camada de Conexão)

Este módulo centraliza a lógica de comunicação em rede. Tipicamente, em aplicações de chat, este componente:

- inicializa e configura o *socket* (endereço IP, porta, modo de bloqueio, etc.);
- estabelece a conexão com o ponto remoto (servidor) e gerencia a vida útil da sessão;
- implementa a leitura e escrita de mensagens, incluindo regras de enquadramento (por exemplo, por terminador de linha `\n` ou prefixo de tamanho);
- trata erros de E/S (timeouts, desconexões, tentativas de reconexão) e o encerramento limpo.

2.2 `client.py` (Aplicação Cliente)

O cliente provê a interface de uso via linha de comando para o usuário final. Entre suas funções típicas estão:

- coleta de parâmetros de execução (endereço do servidor, porta e, opcionalmente, apelido/usuário);
- envio de mensagens digitadas para a conexão ativa;
- recepção e exibição de mensagens de outros participantes;
- tratamento de comandos locais (por exemplo, `/sair`) e finalização graciosa.

3 Funcionalidades Principais

Com base no propósito do repositório (“Projeto de chat em Python”) e na divisão em módulos, destacam-se as seguintes funcionalidades esperadas de um chat de rede:

1. Conexão de um ou mais clientes a um ponto central (servidor) por meio de *sockets*;
2. Envio e recebimento de mensagens de texto entre participantes;
3. Identificação simples do usuário (por apelido) ao ingressar na sessão;
4. Exibição em tempo real das mensagens recebidas;
5. Encerramento correto da sessão de usuário e liberação de recursos.

4 Protocolos Implementados

Aplicações de chat acadêmicas em Python frequentemente utilizam o protocolo TCP (Transmission Control Protocol) na camada de transporte por garantir:

- entrega confiável (retransmissão em caso de perdas);
- ordenação dos segmentos recebidos;
- controle de fluxo e de congestionamento;
- comunicação orientada à conexão.

No nível da aplicação, é comum definir um protocolo simples de mensagens em texto (codificação UTF-8), com uma das formas de enquadramento:

- terminador de linha: cada mensagem termina com `\n`;
- prefixo de tamanho: cada mensagem inicia com o tamanho em bytes seguido do corpo;
- delimitadores especiais (por exemplo, `<MSG> ... </MSG>`).

5 Dificuldades Encontradas

Durante o desenvolvimento de aplicações de chat em rede, são comuns as seguintes dificuldades:

- Concorrência de leitura/escrita (*deadlocks* e condições de corrida);
- Enquadramento de mensagens (garantir leitura de mensagens completas);
- Tratamento de desconexões abruptas e *timeouts*;
- Codificação de caracteres e compatibilidade entre sistemas;

6 O que Poderia Ter Sido Implementado a Mais

Como extensões e melhorias futuras:

- Autenticação de usuário e histórico persistente de mensagens;
- Salas de chat (*rooms*) e mensagens privadas (*DM*);
- Criptografia ponta a ponta ou TLS entre cliente e servidor;
- Protocolo de aplicação estruturado (JSON/MsgPack/Protobuf) com *versionamento*;
- Interface gráfica (GUI) ou cliente web;
- Testes automatizados e integração contínua.

7 Conclusão

Este relatório apresentou a visão geral do projeto de chat em Python, os módulos presentes no repositório e os conceitos de redes envolvidos, com ênfase no uso de *sockets* e na definição de um protocolo de aplicação simples. A separação entre a camada de conexão (`server.py`) e a lógica do cliente (`client.py`) favorece a clareza e a manutenibilidade do código. O conjunto de testes e as melhorias propostas oferecem caminhos para evolução futura do sistema.

Repositório do Projeto

GitHub: levisilva22/projeto-redes

Apêndice A — Impressão do Código-Fonte

```
1 import socket
2 import threading
3 from datetime import datetime
4
5 class ChatServer:
6     def __init__(self, host='0.0.0.0', port=5555):
7         self.host = host
8         self.port = port
9         self.server_socket = None
10        self.clients = {} # {conn: {"username": "nome", "address":
11                           addr}}
12        self.running = False
13
14    def start_server(self):
15        """Inicia o servidor de chat"""
16        self.server_socket = socket.socket(socket.AF_INET, socket.
17        SOCK_STREAM)
18        self.server_socket.setsockopt(socket.SOL_SOCKET, socket.
19        SO_REUSEADDR, 1)
20        self.server_socket.bind((self.host, self.port))
21        self.server_socket.listen()
22
23        self.running = True
24        print(f"[SERVIDOR] Ouvindo em {self.host}:{self.port}")
25        print("[SERVIDOR] Aguardando conex es...")
26
27        try:
28            while self.running:
29                conn, addr = self.server_socket.accept()
30                client_thread = threading.Thread(
31                    target=self.handle_client,
32                    args=(conn, addr)
33                )
34                client_thread.daemon = True
35                client_thread.start()
36        except KeyboardInterrupt:
37            self.shutdown()
38
39    def handle_client(self, conn, addr):
40        """Gerencia a comunica o com um cliente espec fico"""
41        try:
42            # Solicita nome de usu rio
43            conn.send("NOME".encode('utf-8'))
44            username = conn.recv(1024).decode('utf-8').strip()
45
46            if not username:
47                username = f"Usuario_{addr[1]}"
48
49            # Adiciona cliente lista
```

```

47         self.clients[conn] = {
48             "username": username,
49             "address": addr
50         }
51
52         # Notifica todos sobre novo usu rio
53         self.broadcast(f"[SISTEMA] {username} entrou no chat!",
54                        conn)
55         print(f"[CONEX 0] {username} ({addr}) conectou-se.
56               Total: {len(self.clients)}")
57
58         conn.send(f"[SISTEMA] Bem-vindo(a) ao chat, {username}!
59                  ".encode('utf-8'))
60
61         # Loop principal de mensagens
62         while self.running:
63             message = conn.recv(1024).decode('utf-8')
64
65             if not message:
66                 break
67
68             if message.upper() == '/SAIR':
69                 break
70             elif message.upper() == '/USUARIOS':
71                 users = ", ".join([client["username"] for
72                                     client in self.clients.values()])
73                 conn.send(f"[SISTEMA] Usu rios online: {users}
74                           ".encode('utf-8'))
75             else:
76                 # Encaminha mensagem para todos
77                 formatted_msg = f"[{datetime.now().strftime('%H
78                               :%M')}] {username}: {message}"
79                 self.broadcast(formatted_msg, conn)
80                 print(formatted_msg)
81
82         except Exception as e:
83             print(f"[ERRO] Com cliente {addr}: {e}")
84         finally:
85             self.remove_client(conn, username)
86
87     def broadcast(self, message, sender_conn=None):
88         """Envia mensagem para todos os clientes, exceto o
89           remetente"""
90         disconnected_clients = []
91
92         for client_conn in self.clients.keys():
93             if client_conn != sender_conn:
94                 try:
95                     client_conn.send(message.encode('utf-8'))
96                 except:
97                     disconnected_clients.append(client_conn)

```

```

91
92     # Remove clientes desconectados
93     for client in disconnected_clients:
94         self.remove_client(client)
95
96     def remove_client(self, conn, username=None):
97         """Remove cliente da lista e notifica os demais"""
98         if conn in self.clients:
99             if not username:
100                 username = self.clients[conn]["username"]
101
102                 del self.clients[conn]
103                 try:
104                     conn.close()
105                 except:
106                     pass
107
108                 leave_msg = f"[SISTEMA] {username} saiu do chat."
109                 self.broadcast(leave_msg)
110                 print(leave_msg)
111                 print(f"[INFO] Total de usu rios: {len(self.clients)}")
112
113     def shutdown(self):
114         """Encerra o servidor graciosamente"""
115         print("\n[SERVIDOR] Encerrando...")
116         self.running = False
117         self.broadcast("[SISTEMA] Servidor est  sendo encerrado!")
118
119         for conn in list(self.clients.keys()):
120             self.remove_client(conn)
121
122         if self.server_socket:
123             self.server_socket.close()
124         print("[SERVIDOR] Encerrado.")
125
126 if __name__ == "__main__":
127     server = ChatServer()
128     server.start_server()

```

Listing 1: Arquivo server.py

```

1 import socket
2 import threading
3
4 class ChatClient:
5     def __init__(self, host='localhost', port=5555):
6         self.host = host
7         self.port = port
8         self.client_socket = None
9         self.running = False
10        self.username = ""

```



```

11
12 def start_client(self):
13     """Inicia o cliente de chat"""
14     try:
15         self.client_socket = socket.socket(socket.AF_INET,
16                                             socket.SOCK_STREAM)
17         self.client_socket.connect((self.host, self.port))
18
19         # Recebe solicita o de nome
20         message = self.client_socket.recv(1024).decode('utf-8')
21         if message == "NOME":
22             self.username = input("Digite seu nome de usu rio: ")
23             self.client_socket.send(self.username.encode('utf-8'))
24
25         self.running = True
26
27         # Thread para receber mensagens
28         receive_thread = threading.Thread(target=self.receive_messages)
29         receive_thread.daemon = True
30         receive_thread.start()
31
32         # Loop principal para enviar mensagens
33         self.send_messages()
34
35     except Exception as e:
36         print(f"[ERRO] N o foi poss vel conectar ao servidor: {e}")
37     finally:
38         self.disconnect()
39
40 def receive_messages(self):
41     """Recebe mensagens do servidor em uma thread separada"""
42     while self.running:
43         try:
44             message = self.client_socket.recv(1024).decode('utf-8')
45
46             if not message:
47                 print("[SISTEMA] Conex o com o servidor perdida.")
48                 self.running = False
49                 break
50             else:
51                 print(message)
52
53         except:
54             print("[SISTEMA] Erro ao receber mensagens.")
55             self.running = False

```

```

55         break
56
57     def send_messages(self):
58         """Envia mensagens para o servidor"""
59         print("\nComandos dispon veis:")
60         print("/sair - Sair do chat")
61         print("/usuarios - Listar usu rios online")
62         print("Digite sua mensagem e pressione Enter para enviar\n"
63             )
64
65         while self.running:
66             try:
67                 message = input()
68
69                 if not self.running:
70                     break
71
72                 if message.upper() == '/SAIR':
73                     self.client_socket.send('/SAIR'.encode('utf-8'))
74                     break
75                 elif message.upper() == '/USUARIOS':
76                     self.client_socket.send('/USUARIOS'.encode('utf-8'))
77                 else:
78                     self.client_socket.send(message.encode('utf-8'))
79
80             except KeyboardInterrupt:
81                 print("\n[SISTEMA] Saindo...")
82                 break
83             except Exception as e:
84                 print(f"[ERRO] Erro ao enviar mensagem: {e}")
85                 break
86
87     def disconnect(self):
88         """Desconecta do servidor"""
89         self.running = False
90         if self.client_socket:
91             self.client_socket.close()
92             print("[SISTEMA] Desconectado.")
93
94 if __name__ == "__main__":
95     # Permite especificar o IP do servidor
96     server_ip = input("Digite o IP do servidor (ou Enter para localhost): ").strip()
97     if not server_ip:
98         server_ip = 'localhost'
99
100     client = ChatClient(host=server_ip)
101     client.start_client()

```

Listing 2: Arquivo `client.py`